← Índice de la Bible

version: 1.0.0 · **estado**: Aprobado · **fecha**: 2026-07-18 · **fecha_aprobacion**: 2026-07-18 · **autor**: Opus (Chief Software Architect) — aprobación CTO por delegación D22 · **responsable**: Opus · **prioridad**: P1

3.3 — Núcleo Event-Driven y Contratos Internos

> **Estado:** Aprobado (CTO por delegación D22, 2026-07-18) · **Prioridad:** P1 · **Responsable:** Opus

Convención † (1.1 §4): desarrollo del CTO/arquitecto exigible con la aprobación; no es texto de actas del CEO.

Resumen ejecutivo

Especifica el sistema nervioso de AURA: los **contratos** del Event Bus como plano de control (3.2 §4.1). Define la taxonomía y el esquema versionado de eventos, las garantías de entrega (**at-least-once + deduplicación por idempotencia**), el tratamiento especial de **eventos con efectos físicos**, el **invariante de correlación Trace ID** que une bus y streams (resolución ALERTA-001), el manejo de errores con **dead-letter queue**, y los contratos de API internos entre módulos. No elige el broker concreto: esa es una selección de tecnología que se decide en 3.6 (4 niveles, D3). Ejecuta la arquitectura del 3.2 sin cambiar el alcance.

Objetivo

Fijar los contratos que hacen del bus un plano de control fiable, auditable y trazable, de modo que cualquier módulo o plugin se acople a él por interfaz estable (ADR-0004) y que los eventos con efecto físico nunca se dupliquen ni se pierdan.

1. Alcance y frontera con 3.6

Este capítulo define **contratos y patrones** (independientes de producto). La **selección del broker** (in-process, NATS, MQTT interno, Redis Streams, etc.) es una decisión de tecnología reservada al CEO y se presenta en 3.6 con 4 niveles (D3, P-ARQ-6). Los contratos aquí valen para cualquier broker que los cumpla (P-ARQ-4).

2. Taxonomía y esquema de eventos

- **Familias de eventos:** `perception.*` (fin de turno, transcripción, estado de dispositivo), `cognition.*` (intención, decisión de enrutamiento, operación de memoria), `permission.*` (solicitud, autorización, denegación), `action.*` (comando, resultado), `audit.*` (traza inmutable), `system.*` (salud, degradación).
- **Sobre de evento (contrato común):** `event_id` (único), `type`, `schema_version` (SemVer), `trace_id` (correlación §4), `user_id` (multiusuario, 3.2 §7), `timestamp`, `source`, `payload`, `causation_id` (evento que lo causó).
- **Schema registry:** cada `type` tiene un esquema versionado (JSON Schema). El registro es la fuente de verdad de los contratos de evento; su evolución sigue la **regla 11 del ADR-0004** (SemVer; ruptura → ADR + máximo 2 versiones mayores vivas; convivencia con término).

3. Garantías de entrega e idempotencia

- **At-least-once** como default: un consumidor puede recibir un evento más de una vez.
- **Idempotencia obligatoria en efectos físicos** (checklist 3.1 §2.3 Q9): todo consumidor de `action.*` con efecto físico o irreversible deduplica por `event_id`; ejecutar el mismo comando dos veces produce un solo efecto. Es el mecanismo contra la duplicación de acciones domóticas/sistema (riesgo grave, 3.1).
- **Orden por `trace_id`:** dentro de una misma interacción, los eventos se procesan en orden causal (`causation_id`); entre interacciones no se garantiza orden global (innecesario y costoso).
- **Exactly-once no se promete** a nivel de transporte (falacia distribuida); se logra *efectivamente* con at-least-once + idempotencia, más barato y sostenible (regla 13 ADR-0004).

4. Invariante de correlación bus ↔ streams (ALERTA-001)

Materializa el invariante que el 3.2 §4.4 declaró: todo stream de baja latencia (voz, avatar) **se abre y se cierra con eventos de control en el bus que comparten su `trace_id`**. Contrato:

- Al iniciar un turno de voz, el Adaptador de Voz emite `perception.turn_start` con un `trace_id` nuevo; el audio fluye por el canal dedicado etiquetado con ese mismo `trace_id`.
- Toda decisión y acción derivada hereda el `trace_id` (via `causation_id`).
- Al cerrar, `perception.turn_end` con el mismo `trace_id`.
- **Sin `trace_id` no se admite tráfico de stream:** un stream huérfano se rechaza y se registra en `audit.*` (garantiza P-ARQ-1/7).

5. Errores, reintentos y dead-letter

- **Reintentos con backoff** exponencial acotado por consumidor; el número máximo es configuración, no código (regla 12 ADR-0004).
- **Dead-letter queue (DLQ):** un evento que agota reintentos va a la DLQ con su causa; nunca se descarta en silencio (P-ARQ-1). La DLQ es local y auditable.
- **Efectos físicos fallidos:** un `action.*` con efecto físico que falla emite `action.result` con estado de error y **no** se reintenta automáticamente si el efecto es irreversible; escala a HITL (5.3).

6. Contratos de API internos

Entre módulos que no se comunican por eventos (llamada síncrona: p. ej. el orquestador consulta memoria) rige un contrato de API interno: interfaz versionada (SemVer), modelo de errores unificado, y validación de esquema en el borde. Igual que los eventos, su ruptura exige ADR (regla 11). El orquestador nunca conoce la *implementación* de un módulo, solo su contrato (P-ARQ-3).

Decisiones tomadas

- Contratos de evento, sobre común, schema registry, garantías at-least-once + idempotencia, invariante Trace ID, DLQ y contratos de API internos: ejecución de la arquitectura del 3.2 (desarrollo del arquitecto †, aprobado por delegación).
- Selección del broker concreto: **diferida a 3.6** (tecnología, D3).

Decisiones aplicables (ADRs)

- ADR-0004 — Evolución Permanente (`accepted`): reglas 11 (versionado/deprecación de contratos) y 12 (config, no código) gobiernan este capítulo.
- ADR-0001 — Local-first (`accepted`): el bus y la DLQ son locales; los eventos `audit.*` son dato del usuario.

Evaluación por las cinco preguntas (D21)

(1) Escala 10 años: esquema versionado admite tipos nuevos sin romper. (2) Reduce dependencia: contratos independientes del broker. (3) Sustituir sin rediseñar: cambiar de broker no toca a los consumidores. (4) Equipo pequeño: at-least-once+idempotencia evita la complejidad de exactly-once. (5) Sistema vivo: nuevos módulos se enchufan por evento. Sin respuesta negativa → sin Alerta.

Riesgos

- **Idempotencia mal implementada** en un adaptador físico → acción duplicada; mitigación: es requisito de contrato verificable (checklist 3.1 §2.3 Q9) y se prueba en 7.2.
- **Trace_id perdido** → traza rota; mitigación: invariante §4, stream huérfano rechazado.
- **DLQ que crece sin vigilancia** → fallos silenciosos acumulados; mitigación: la DLQ es observable (P-ARQ-7) y su revisión entra en la operación mínima (7.4).

Dependencias

Capítulos: **3.2** (Aprobado). Alimenta: 3.4 (los plugins consumen/emiten por estos contratos), 3.6 (elige el broker), 4.1-4.4 (módulos que se comunican por el bus), 5.3 (HITL sobre `action.*`), 7.2 (pruebas de idempotencia y correlación).

Próximos pasos

- 3.6 presenta al CEO la selección del broker (4 niveles).
- 3.4 define el contrato de plugin sobre esta base de eventos.

Referencias cruzadas

- 3.2 Arquitectura de Referencia · 3.1 Principios · ALERTA-001
- ADR-0001 · ADR-0004 · Índice maestro

Aprobación del CTO (delegación D22)

Revisión ejecutiva del CTO conforme al acta D22: **(1) Alcance** — el capítulo ejecuta contratos ya delegados por el 3.2 (Aprobado); no añade capacidades ni cambia el alcance del MVP (2.2). **(2)**

Consistencia — respeta ADR-0001, ADR-0004 (reglas 11/13), los principios P-ARQ (3.1) y la resolución de la ALERTA-001. **(3) Cinco preguntas (D21)** — todas afirmativas (ver sección). **(4) Materia reservada al CEO** — ninguna: las selecciones de producto concreto se difieren al 3.6 (4 niveles, D3). **Veredicto: APROBADO por el CTO (delegación D22), 2026-07-18.**
